



APIs and GraphQL

Building and Consuming Web APIs

Chapter Overview

- 01** REST API Fundamentals and Fetch
- 02** Async Patterns and Error Handling
- 03** React Data Fetching Patterns
- 04** Introduction to GraphQL
- 05** Advanced GraphQL and API Security



REST API Fundamentals and Fetch

Section 1



What is a REST API?

Representational State Transfer

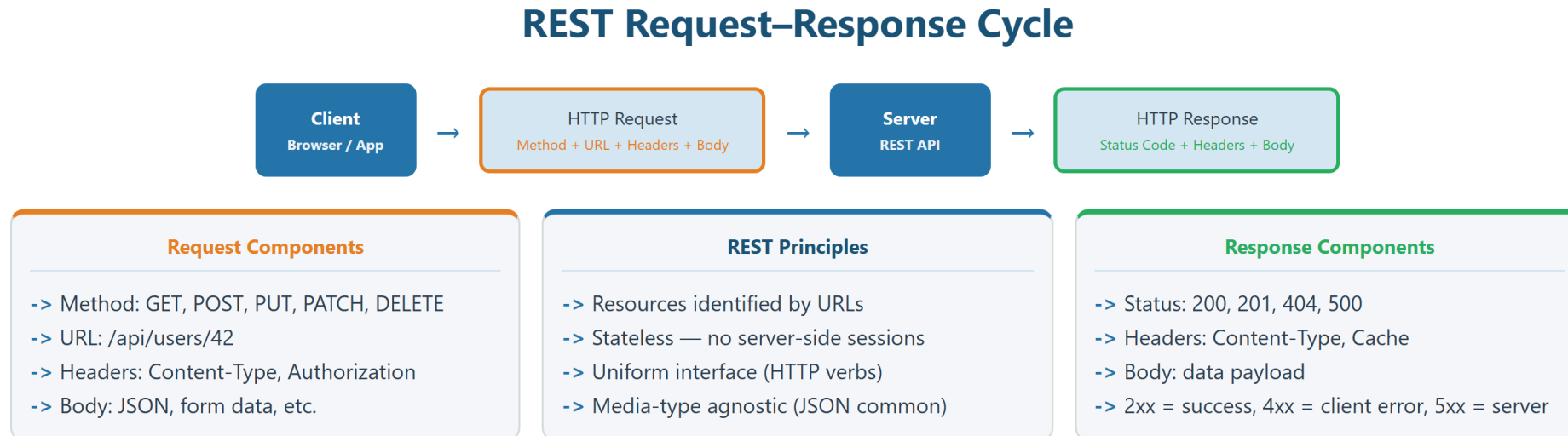
- An architectural style for building web APIs around resources
- Uses standard HTTP methods to perform operations on resources

Core REST Constraints

- Resources identified by URLs — /api/users, /api/users/42
- Stateless — each request carries all information the server needs
- Uniform interface — standard HTTP verbs for all operations
- Media-type agnostic — JSON is the most common format, but not a REST requirement



REST Request–Response Cycle



HTTP Methods in Detail

CRUD Mapping

- GET — Read a resource or collection (safe, idempotent)
- POST — Create a new resource (not idempotent)
- PUT — Replace an entire resource (idempotent)
- PATCH — Partially update specific fields of a resource (not necessarily idempotent)
- DELETE — Remove a resource (idempotent)

PUT vs PATCH

- PUT sends the complete replacement; PATCH sends only the changed fields



HTTP Status Codes

Success Codes (2xx)

- 200 OK — request succeeded
- 201 Created — new resource created (POST)
- 204 No Content — success with no body (DELETE)

Client Error Codes (4xx)

- 400 Bad Request — malformed input
- 401 Unauthorized — authentication required
- 403 Forbidden — authenticated but not allowed
- 404 Not Found — resource does not exist

Server Error Codes (5xx)

- 500 Internal Server Error — unhandled server failure



The Fetch API — GET and POST

```
// GET request — read data
const response = await fetch('https://api.example.com/users');
const users = await response.json();

// POST request — create data
const created = await fetch('https://api.example.com/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    name: 'Alice',
    email: 'alice@example.com'
  })
});
const newUser = await created.json();
```



Configuring Fetch Requests

Request Options Object

- method — GET (default), POST, PUT, PATCH, DELETE
- headers — Content-Type, Authorization, custom headers
- body — `JSON.stringify(data)` for POST/PUT/PATCH requests
- signal — `AbortController` signal for cancellation

The Response Object

- `response.ok` — true if status is 200–299
- `response.status` — the numeric HTTP status code
- `response.json()` — returns a Promise that resolves to parsed JSON



Async Patterns and Error Handling

Section 2



How Fetch Works Asynchronously

Two Awaits Required

- First await — waits for the HTTP response headers to arrive
- Second await — waits for the response body to be fully read and parsed
- Each step returns a Promise that can be awaited or chained with `.then()`

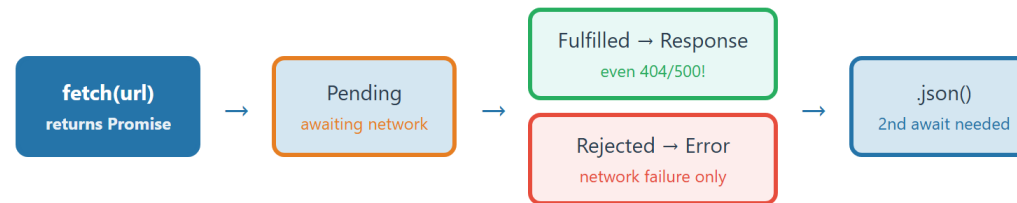
Critical Gotcha

- `fetch()` does NOT reject on HTTP errors (404, 500)
- Only network failures (DNS errors, offline, CORS blocks) cause rejection
- You must check `response.ok` before processing the body



Fetch Async Data Flow

Fetch API — Async Data Flow



⚠ Critical Gotcha

fetch() does NOT throw on HTTP errors.

A 404 or 500 response still *fulfills* the promise.

Only network failures (DNS, offline) cause rejection.

You **must** check `response.ok` manually.

Two-Step Async Pattern

```
// Step 1: await the network
const res = await fetch(url);

// Step 2: await body parsing
const data = await res.json();
```

fetch() is natively available in all modern browsers and server-side runtimes (Node.js 18+, Deno, Bun)

Error Handling with Fetch

The Correct Pattern

- Wrap fetch calls in try/catch to handle network errors
- Inside try, check response.ok — throw for non-2xx status codes
- Catch block handles both network failures and HTTP errors

Presenting Errors to Users

- Display user-friendly messages — not raw status codes or stack traces
- Provide a retry option for transient errors
- Log detailed error info to the console for debugging



Robust Fetch Pattern

```
async function fetchJSON(url, options = {}) {
  try {
    const response = await fetch(url, options);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
    return await response.json();
  } catch (error) {
    if (error.name === 'AbortError') {
      console.log('Request was cancelled');
      return null;
    }
    console.error('Fetch failed:', error.message);
    throw error; // re-throw for caller to handle
  }
}
```



Parallel and Sequential Requests

Sequential Requests

- `const user = await fetchJSON('/api/users/1')`
- `const posts = await fetchJSON(`/api/users/${user.id}/posts`)`
- Use when the second request depends on the first result

Parallel Requests with `Promise.all`

- Fires all requests simultaneously — much faster for independent data
- Rejects immediately if any single request fails

`Promise.allSettled`

- Waits for ALL requests — returns `{status, value/reason}` for each



Promise.all and Promise.allSettled

```
// Parallel – fast, but fails if any request fails
const [users, posts, comments] = await Promise.all([
  fetchJSON('/api/users'),
  fetchJSON('/api/posts'),
  fetchJSON('/api/comments'),
]);

// Resilient – always completes, reports per-request status
const results = await Promise.allSettled([
  fetchJSON('/api/users'),
  fetchJSON('/api/posts'),
]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.error(r.reason);
});
```



Loading States and UX Best Practices

Three States of a Data Fetch

- Loading — show a spinner, skeleton screen, or progress indicator
- Success — render the data
- Error — show a message with a retry button

Loading Indicator Guidelines

- Show indicators for requests that may take noticeable time (varies by context)
- Use skeleton screens for content-heavy layouts to reduce perceived load time
- Disable submit buttons while a request is in flight to prevent duplicates



React Data Fetching Patterns

Section 3



Data Fetching in React with useEffect

Why useEffect?

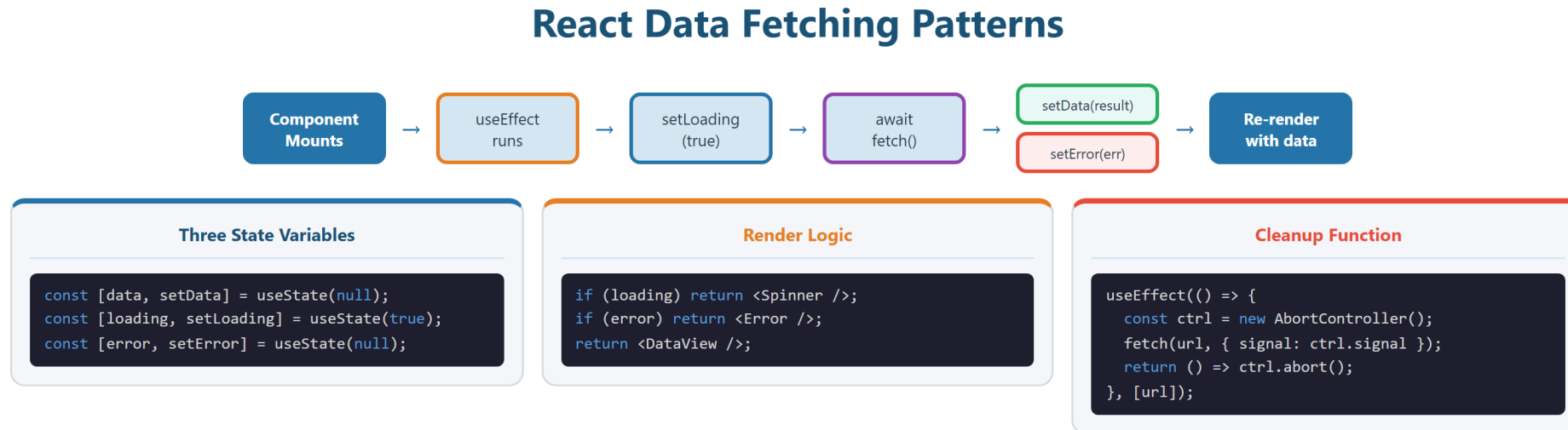
- React components are pure functions — side effects like fetching go in useEffect
- useEffect runs after the component renders (not during)
- The dependency array controls when the effect re-runs

The Data Fetching Pattern

- Define an async function inside useEffect (effects cannot be async directly)
- Call the async function immediately
- Store results in state via useState — triggers re-render with data



React Data Fetching Patterns



Production apps often use framework data loaders (Next.js, Remix) or libraries (React Query / SWR) instead of raw useEffect

Basic useEffect Fetch Pattern

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadUsers() {
      try {
        const res = await fetch('/api/users');
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        setUsers(await res.json());
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    loadUsers();
  }, []); // empty array = run once on mount
}
```



Managing Loading, Error, and Data State

Three State Variables

- data (null → array/object) — the fetched payload
- loading (true → false) — controls spinner visibility
- error (null → string) — holds the error message if the request failed

Conditional Rendering Pattern

- if (loading) return <Spinner /> — show loading indicator first
- if (error) return <ErrorMessage /> — show error with retry button
- return <DataView data={data} /> — render the actual content last



Complete Data Fetching Component

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // ... useEffect fetch logic from previous slide ...

  if (loading) return <p>Loading users...</p>;
  if (error) return (
    <div>
      <p>Error: {error}</p>
      <button onClick={() => window.location.reload()}>Retry</button>
    </div>
  );
  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```



Cleanup and Race Conditions

The Problem

- If a component unmounts while a fetch is in flight, updating state on an unmounted component wastes resources and caused a warning in React <18
- Rapid dependency changes can cause out-of-order responses (race condition)

The Solution: AbortController

- Create an AbortController and pass its signal to fetch()
- Return a cleanup function from useEffect that calls controller.abort()
- Catch AbortError in the error handler and ignore it — it's intentional



useEffect Cleanup with AbortController

```
useEffect(() => {
  const controller = new AbortController();

  async function fetchData() {
    try {
      setLoading(true);
      const res = await fetch(`/api/users/${id}`, {
        signal: controller.signal,
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setData(await res.json());
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message);
    } finally {
      setLoading(false);
    }
  }
  fetchData();

  return () => controller.abort(); // cleanup on unmount or id change
}, [id]);
```



Introduction to GraphQL

Section 4



What is GraphQL?

A Query Language for APIs

- Developed by Facebook in 2012, open-sourced in 2015
- Single endpoint (/graphql) replaces many REST endpoints
- Client specifies exactly which fields it needs — no more, no less

Key Advantages Over REST

- Reduces over-fetching — request only the fields your UI displays
- Reduces under-fetching — get related data in a single request
- Strongly typed schema — the API is self-documenting
- Excellent for multiple client types (web, mobile) needing different data



GraphQL vs REST

GraphQL vs REST — Key Differences

REST API

Multiple Endpoints

```
GET /api/users/42  
GET /api/users/42/posts  
GET /api/users/42/followers
```

- > 3 requests for related data
- > Server decides response shape
- > Over-fetching: get ALL user fields
- > Under-fetching: need extra calls
- > Simple CRUD operations
- > Cacheable via HTTP caching

GraphQL

Single Endpoint

```
POST /graphql  
query { user(id: 42) {  
  name, posts { title },  
  followers { name }  
} }
```

- > 1 request for all related data
- > Client specifies exact fields
- > Reduces over-fetching and under-fetching
- > Strongly typed schema
- > Complex, relationship-heavy data
- > Needs client-side caching (Apollo)

Both approaches are valid — choose based on data complexity and client needs

GraphQL Schema and Type System

The Schema Definition Language (SDL)

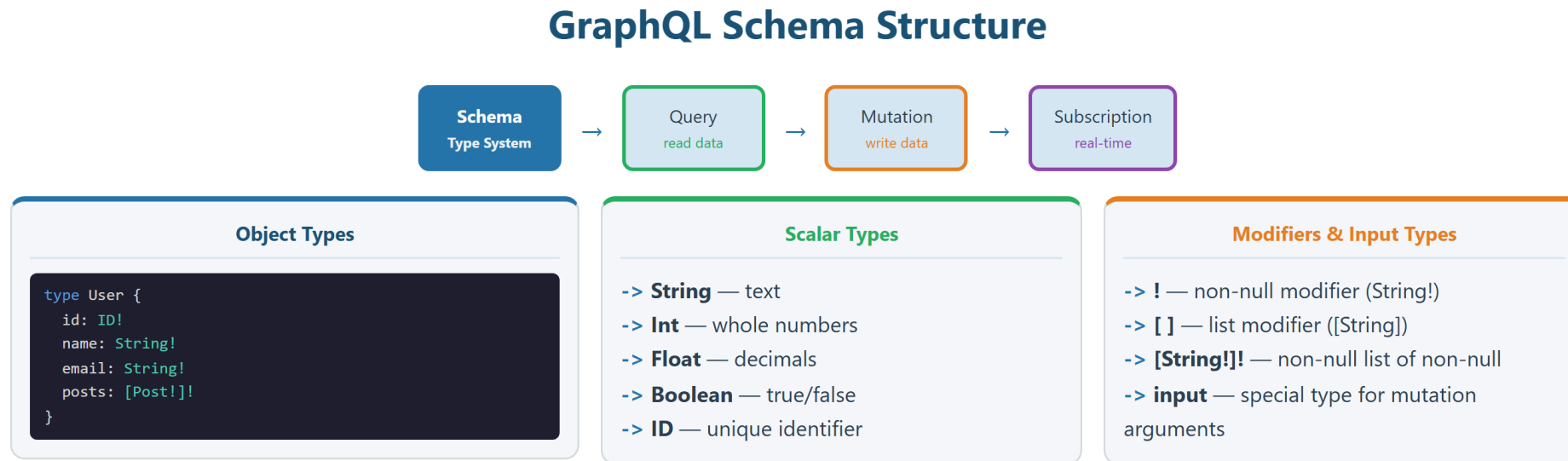
- Defines every type, field, and relationship in the API
- Scalar types: String, Int, Float, Boolean, ID
- ! means non-null — String! cannot return null

Three Root Operation Types

- Query — read data (analogous to GET)
- Mutation — write data (create, update, delete operations)
- Subscription — receive real-time updates via WebSocket



GraphQL Schema Structure



The schema serves as a contract between client and server — self-documenting API

Your First GraphQL Query

```
# REST approach: GET /api/users/42 returns ALL fields
# GraphQL approach: request exactly what you need

query GetUserWithPosts {
  user(id: 42) {
    name
    email
    posts {
      title
      createdAt
    }
    followers {
      name
    }
  }
}
```

One request replaces 3 REST calls (user + posts + followers)



Variables and Fragments

Query Variables

- Use \$variables to parameterize queries — never concatenate user input
- Variables are passed as a separate JSON object alongside the query
- Typed: `query GetUser($id: ID!) { user(id: $id) { name } }`

Fragments — Reusable Field Sets

- `fragment UserFields on User { name email avatar }`
- Use with spread: `{ user(id: 1) { ...UserFields } }`
- Reduce duplication when the same fields are queried in multiple places



Fetching GraphQL from JavaScript

```
const QUERY = `
  query GetUser($id: ID!) {
    user(id: $id) {
      name
      email
      posts { title }
    }
  }
`;

const response = await fetch('/graphql', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    query: QUERY,
    variables: { id: '42' }
  })
});
const { data, errors } = await response.json();
```



The GraphQL Error Model

Partial Responses

- GraphQL can return both data AND errors in the same response
- HTTP 200 is returned with an errors array for resolver-level issues; transport/auth failures may still return 4xx/5xx
- Always check the errors array in the response body

Error Structure

- Each error has: message, locations (line/column), path (field path), extensions
- Extensions can carry custom error codes (UNAUTHENTICATED, NOT_FOUND)
- Client must check response.data AND response.errors separately



Advanced GraphQL and API Security

Section 5



GraphQL Mutations

Writing Data with Mutations

- Mutations are GraphQL's mechanism for creating, updating, and deleting data
- Unlike REST, there is no direct mapping to POST/PUT/DELETE — the mutation name defines the operation
- Mutations return data, so you can get the updated resource immediately

Best Practices

- Use input types for complex arguments: mutation CreateUser(\$input: CreateUserInput!)
- Always use variables — never interpolate user input into queries
- Return enough fields to update your client-side cache



Mutation with Variables

```
const CREATE_USER = `
  mutation CreateUser($input: CreateUserInput!) {
    createUser(input: $input) {
      id
      name
      email
    }
  }
`;

const { data } = await fetch('/graphql', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`,
  },
  body: JSON.stringify({
    query: CREATE_USER,
    variables: { input: { name: 'Alice', email: 'a@b.com' } }
  })
}).then(r => r.json());
```



Subscriptions and Real-Time Data

What Are Subscriptions?

- GraphQL's built-in mechanism for real-time updates from the server
- Uses WebSocket protocol instead of HTTP request-response
- Client subscribes to events; server pushes data as it changes

Use Cases

- Live chat messages, notifications, real-time dashboards
- Collaborative editing — see other users' changes immediately

Syntax

- `subscription { messageAdded(roomId: "123") { text author } }`



Authentication, CORS, and API Security

Token-Based Authentication

- Login endpoint returns a JWT; prefer httpOnly cookies or in-memory storage over localStorage
- Attach via Authorization: Bearer <token> header on every request
- Works the same for REST and GraphQL APIs

CORS (Cross-Origin Resource Sharing)

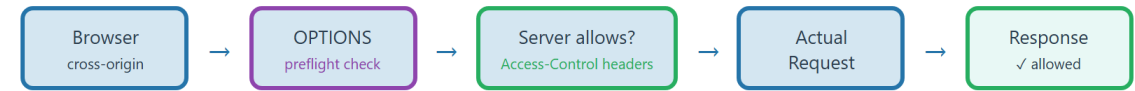
- Browsers block cross-origin requests by default for security
- Server must respond with Access-Control-Allow-Origin for specific origins (avoid wildcard * in production)
- Preflight OPTIONS request sent automatically for non-simple requests



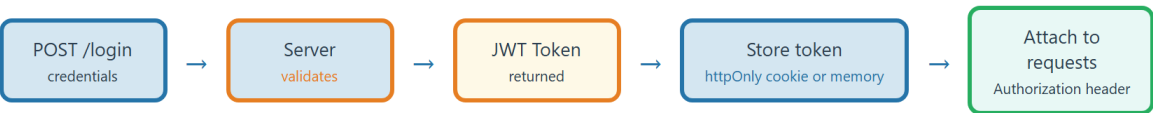
Authentication and CORS

Authentication & CORS

CORS Preflight Flow



Token-Based Authentication



CORS Headers

```
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Authorization
```

Auth Header

```
headers: {
  'Authorization': 'Bearer ' + token,
  'Content-Type': 'application/json'
}
```



Caching, Pagination, and Performance

REST Caching

- Built-in HTTP caching via Cache-Control, ETag, and 304 Not Modified
- Works naturally with CDNs and browser caches

GraphQL Caching

- HTTP caching is less straightforward — all requests are POST; use persisted queries, CDN strategies, or client-side caching
- Client-side caching via libraries like Apollo Client (normalized cache)

Pagination Patterns

- Offset-based: `/api/users?page=2&limit=20` (REST) or `users(offset: 20, limit: 20)`
- Cursor-based: `users(first: 20, after: "cursor")` — better for real-time feeds



Apollo Client and the GraphQL Ecosystem

Why Use a GraphQL Client Library?

- Normalized caching — automatic cache updates when data changes
- React hooks: `useQuery`, `useMutation`, `useSubscription`
- Built-in loading/error state management — no manual `useState` needed

Popular GraphQL Tools

- Apollo Client — most popular, full-featured React integration
- `urql` — lightweight alternative, excellent for smaller projects
- GraphQL Playground / GraphiQL — interactive query explorer for development

Choosing REST vs GraphQL

- REST: simpler, better caching, great for CRUD-heavy apps with few relationships
- GraphQL: better for complex data graphs, multiple client types, and flexible UIs



Key Takeaways

Key Takeaways

1. REST APIs use HTTP methods on resource URLs — PUT replaces, PATCH updates fields
2. `fetch()` is Promise-based and does NOT throw on HTTP errors — always check `response.ok`
3. Use `AbortController` to cancel in-flight requests and prevent race conditions
4. In React, fetch inside `useEffect` with `loading/error/data` state and a cleanup function
5. GraphQL uses a single endpoint with a typed schema — clients request exactly what they need
6. Mutations write data, subscriptions enable real-time — always use variables, never string interpolation

